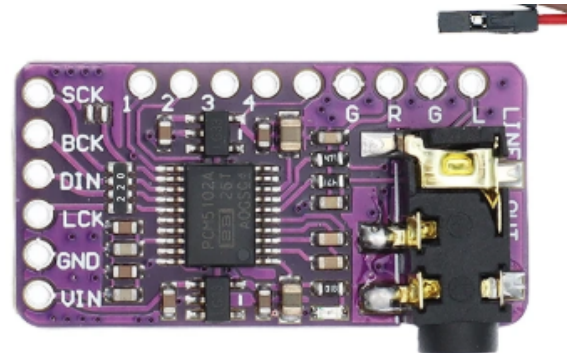
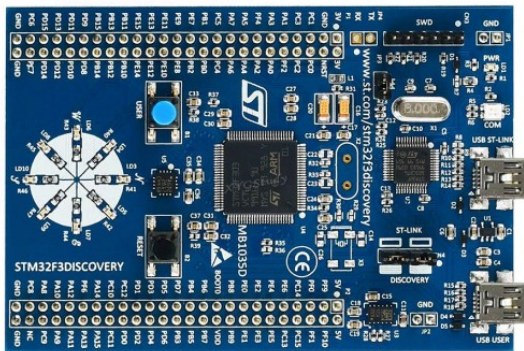


Real-Time Embedded Digital Synthesizer on STM32

A monophonic embedded audio synthesizer



Antonio Maria Improta
Mirko Fanini

Master's Degree in Computer Engineering
University of Naples Federico II

Computer Architecture and Design
Professor Nicola Mazzocca
Academic Year 2024/2025

1 Project overview

The project originally started as the final project for the *Computer Architecture and Design* course, taught by Professor Nicola Mazzocca at the University of Naples Federico II, as part of the Master's Degree in Computer Engineering during the 2024/2025 academic year. The original course version was developed together with my colleague Mirko Fanini. After the exam, I continued and expanded the project with additional audio features, improved diagnostics, signal analysis tools and a more modular software architecture.

This project implements a real-time monophonic digital synthesizer using an STM32F3Discovery board and an external PCM5102A audio DAC. A Novation Launchkey 49 MK3 sends MIDI data to a Python program, which forwards the messages to the microcontroller through a serial connection. The STM32 generates the waveform, applies the DSP chain and continuously transmits the result to the DAC over I2S.

The STM32F3Discovery was selected because it was the development board provided during the course. Its STM32F303VCT6 microcontroller is also suitable for this application because it includes DMA, USART, I2S and a hardware floating-point unit.

MIDI keyboard → Python bridge → USART parser
 → DDS oscillator → ADSR → TPT-SVF
 → 16-bit PCM → DMA/I2S → PCM5102A

The synthesizer provides sine, square and saw waveforms, last-note monophonic priority, an ADSR envelope, a resonant low-pass filter and MIDI control of the main sound parameters. Audio is produced as signed 16-bit PCM at a nominal sample rate of 48 kHz.

2 Hardware architecture

The STM32 works as the I2S master transmitter. The PCM5102A converts the digital samples into a stereo analog output. The same monophonic signal is currently sent to both the left and right channels.

Connection	STM32F3Discovery	PCM5102A	Function
Word select	PB12 / I2S2_WS	LCK / LRCK	Selects the left or right channel
Bit clock	PB13 / I2S2_CK	BCK	Clocks each transmitted audio bit
Audio data	PB15 / I2S2_SD	DIN	Carries the 16-bit PCM samples
Power	5 V	VIN	Powers the DAC breakout board
Ground	GND	GND	Provides a common electrical reference
Clock setting	GND	SCK	Uses the board without an external master clock
Output enable	3.3 V	XSMT / XMT	Keeps the DAC output active

Table 1: Connections between the development board and the PCM5102A module.

3 Real-time audio flow

3.1 MIDI and serial reception

The Python script reads Note On, Note Off and Control Change messages from the Launchkey and sends their raw bytes over a 115200-baud serial link. USART1 on the STM32 receives one byte at a time using an interrupt. The callback stores each byte in a ring buffer and immediately restarts reception. Message parsing stays in the main loop, keeping the interrupt short.

The firmware supports MIDI running status, last-note priority and return to a previously held note. The default knob mapping can be adapted to another controller by editing one table in `midi_mapping.c`.

3.2 DMA and double buffering

The output buffer contains interleaved stereo samples:

```
left[0], right[0], left[1], right[1], ...
```

DMA1 Channel 5 transfers the buffer to I2S2 in circular mode. The buffer is divided into two halves. When DMA finishes transmitting the first half, an interrupt asks the CPU to refill it while DMA continues with the second half. The same process is repeated in the opposite direction at the end of the transfer. This double-buffering method provides a continuous audio stream while leaving the CPU free to calculate the next block.

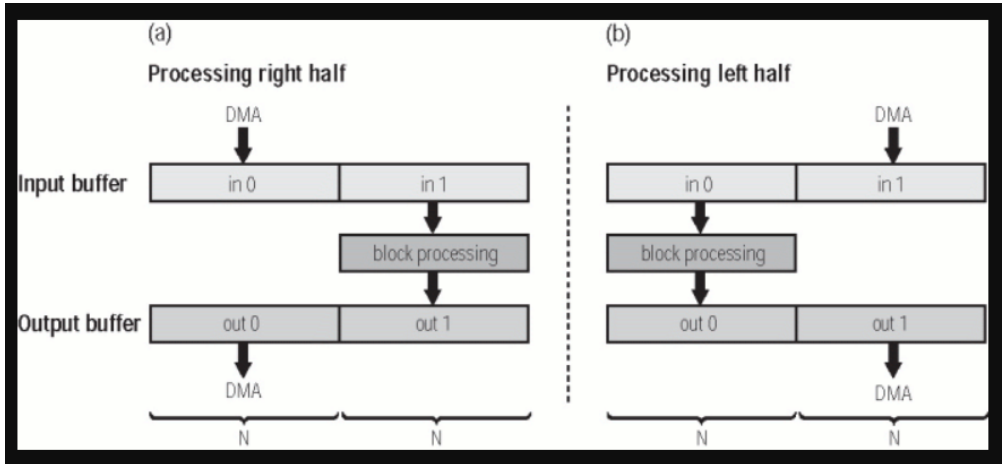


Figure 1: Basic double-buffering process used for continuous DMA transfers.

4 DSP implementation

4.1 Direct Digital Synthesis

The oscillator uses Direct Digital Synthesis with a 32-bit phase accumulator. For a requested frequency f_0 , the phase increment is

$$\Delta\phi = \left\lfloor \frac{f_0 2^{32}}{f_s} \right\rfloor, \quad (1)$$

where f_s is the sample rate. At each sample, the accumulator is updated as

$$\phi[n] = \phi[n-1] + \Delta\phi \pmod{2^{32}}. \quad (2)$$

The ten most significant bits select one of 1024 wavetable samples. The lower 22 bits describe the position between adjacent entries and can be used for linear interpolation:

$$y = x_i + \mu(x_{i+1} - x_i), \quad 0 \leq \mu < 1. \quad (3)$$

Interpolation reduces the error introduced by the finite table resolution. It improves table reading but does not remove aliasing by itself.

4.2 Mipmapped wavetable anti-aliasing

An ideal saw or square wave contains harmonics that extend beyond the Nyquist frequency. If those harmonics are generated digitally, they fold back into the audible range as aliasing.

The project addresses this problem with band-limited mipmapped wavetables. Nine versions of each saw and square waveform are stored in Flash, with maximum harmonic orders

$$511, 255, 127, 63, 31, 15, 7, 3, 1.$$

For a fundamental frequency f_0 , the allowed harmonic order is limited by

$$H_{\max} \leq \left\lfloor \frac{0.9 f_s}{2f_0} \right\rfloor. \quad (4)$$

The factor 0.9 provides a small margin below Nyquist. The firmware selects the richest table that respects this limit. This approach reduces high-frequency aliasing while keeping the processing cost predictable. The tables are generated offline and stored as constant data, so they do not consume working RAM.

4.3 ADSR envelope and low-pass filter

The amplitude envelope is evaluated for every sample and follows the standard attack, decay, sustain and release stages. MIDI CC messages control the four parameters.

The signal then passes through a Topology-Preserving Transform State Variable Filter in low-pass mode. Its main coefficients start from

$$g = \tan\left(\pi \frac{f_c}{f_s}\right), \quad k = \frac{1}{Q}, \quad (5)$$

where f_c is the cutoff frequency and Q controls resonance. Cutoff and Q are smoothed between buffers before the coefficients are updated. This avoids abrupt parameter changes while keeping expensive calculations outside the per-sample loop.

After filtering, the output is limited to the interval $[-1, 1]$, converted to signed 16-bit PCM and written into both stereo channels of the DMA buffer.

5 Real-time performance

The execution time of `FillI2SBuffer()` was measured on the STM32 using the Cortex-M4 DWT cycle counter. Each DMA half-buffer contains 256 stereo frames, so the available processing time is

$$T_{\text{deadline}} = \frac{256}{48000} = 5.333 \text{ ms.} \quad (6)$$

With a 48 MHz CPU clock, this corresponds to

$$C_{\text{deadline}} = 48 \times 10^6 \cdot T_{\text{deadline}} = 256000 \text{ cycles.} \quad (7)$$

During the stress test, the maximum observed execution time was 110994 cycles:

$$T_{\text{max}} = \frac{110994}{48 \times 10^6} = 2.312 \text{ ms.} \quad (8)$$

The resulting maximum observed CPU load was

$$L_{\text{CPU}} = \frac{110994}{256000} \cdot 100 = 43.36\%. \quad (9)$$

Measurement	Result
CPU clock	48 MHz
Frames per DMA half-buffer	256
Half-buffer deadline	5.333 ms
Available cycles	256000
Maximum observed cycles	110994
Maximum observed processing time	2.312 ms
Maximum observed CPU load	43.36%
Remaining timing margin	56.64%
Observed deadline misses	0

Table 2: Real-time performance measured during the firmware stress test.

The test completed without deadline misses and left a measured timing margin of 56.64%. These values represent the maximum observed load during the test and should not be interpreted as a formal worst-case execution-time proof.

6 Software organization

The firmware was divided into small modules so that each DSP block has a clear responsibility:

Module	Responsibility
<code>oscillator.c</code>	DDS phase, waveform generation, interpolation and mipmap selection
<code>oscillator_tables.c</code>	Band-limited waveform data stored in Flash
<code>adsr.c</code>	Per-sample amplitude envelope
<code>svf_filter.c</code>	TPT-SVF low-pass processing and parameter mapping
<code>midi_mapping.c</code>	Configurable association between MIDI CC numbers and synth parameters
<code>main.c</code>	Peripheral setup, MIDI parsing, DMA callbacks and audio-chain orchestration

The oscillator tables are generated by a Python script, while separate analysis scripts reproduce the firmware algorithms and produce frequency-response and spectral plots. This makes it possible to study the DSP behavior before testing the complete hardware chain.